

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

Revelio: A Global Study of VoIP Blocking

Arjun Gupta, *IIT Bombay;* Roya Ensafi, *University of Michigan;*
Piyush Kumar, *IIT Delhi;* Devashish Gosain, *IIT Bombay*

https://www.usenix.org/conference/usenixsecurity26/presentation/gupta

This paper is included in the Proceedings of the
35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the
35th USENIX Security Symposium
is sponsored by



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

Revelio: A Global Study of VoIP Blocking

Arjun Gupta
IIT Bombay

Roya Ensafi
University of Michigan

Piyush Kumar
IIT Delhi

Devashish Gosain
IIT Bombay

Abstract

Online censorship has substantially increased over the past decade. While prior work on censorship detection has largely focused on the web, the landscape and mechanisms of VoIP censorship remain poorly understood.

This work presents the first global study of VoIP blocking by introducing *Revelio*, a system that systematically detects and comprehensively analyzes VoIP censorship at scale. Unlike the only prior work, which is limited to two countries and requires authors to travel to conduct measurements, *Revelio* remotely uncovers VoIP blocking by carefully crafting probe packets and strategically leveraging IP address space within target countries, enabling detection of otherwise invisible VoIP censorship.

Using *Revelio*, we studied 71 countries, covering 11 widely used VoIP applications. Our findings reveal evidence of VoIP blocking in 19 countries, primarily through targeted interference with the STUN traffic. By studying packet traces, we reverse-engineer censorship behavior and uncover four classes and six subtypes of middleboxes that trigger on unique application-specific STUN packets. We further analyze domain-level blocking at the HTTP and HTTPS layers, revealing cases where only VoIP calling is blocked while messaging remains accessible.

Overall, our study advances the understanding of the privacy and censorship community by providing the first comprehensive, global view of an understudied yet highly prevalent form of censorship. We release our code to support reproducibility and future research.

1 Introduction

Internet censorship is on the rise worldwide [31], evolving beyond traditional mechanisms such as web content blocking into more sophisticated and pervasive forms [5,65,70]. While prior work has extensively documented large-scale web censorship [14,23,34,47,50,56,68], recent evidence suggests that censorship increasingly targets real-time communication services as well. In particular, Voice-over-IP (VoIP) based filtering has emerged as a growing and underexplored censorship vector. Anecdotal reports [4,40,60], combined with a recent empirical study in two countries [41], indicate systematic interference with VoIP traffic, raising new concerns about the openness of modern communication infrastructures. Despite the widespread reliance on VoIP services (voice and video calls) for personal, professional, and political communication, little is known about the prevalence, mechanisms, and implications of VoIP censorship in practice.

Studying VoIP censorship in the wild, however, poses substantial methodological challenges. Unlike web traffic, VoIP communication predominantly relies on UDP-based protocols, making it difficult to reliably trigger, observe, and attribute filtering behavior [41]. Network artifacts such as packet loss and congestion further complicate distinguishing intentional interference from benign network impairments. Moreover, an existing empirical study of VoIP filtering [41] relied on in-country vantage points, requiring researchers to physically travel and operate measurement devices within censored regions. Such approaches are costly, difficult to scale, and can expose participants to personal and legal risks. As a result, our understanding of VoIP censorship remains fragmented and limited to a small number of case studies.

This work presents *Revelio*[1], the first system to enable scalable, reliable, large-scale measurement of VoIP censorship. *Revelio* introduces novel detection methods that allow VoIP filtering to be both systematically triggered and robustly identified remotely. At a high level, *Revelio* consists of two core components: a scanner module and a detection module.

The scanner module crafts packets, conducts large-scale scans, and identifies endpoints for testing VoIP censorship remotely in the target country. Since most VoIP apps rely on the STUN/TURN protocol [42] to perform NAT traversal and establish voice and video calls, identifying reachable STUN infrastructure is a critical first step toward measuring VoIP censorship. Notably, existing Internet-wide scanning tools

[1] *Revelio* is a spell from the Harry Potter universe used to reveal concealed objects.

such as `Zmap` [20] fail to identify usable VoIP infrastructure; standard `Zmap` scans discover virtually no open STUN servers in our target regions. We identify the root cause and address this limitation by extending `Zmap` with STUN-aware probing, resulting in `Zstun`, which reliably discovers orders of magnitude more STUN endpoints. Furthermore, the scanner module even leverages the unresponsive IP address space to enable the controlled triggering of VoIP-based filtering (see Section 4.1). To identify potential cases of censorship, the scanner module initially sends two types of probes: (1) vanilla STUN, (2) VoIP app-specific STUN. Upon observing differences in responses, it sends another probe by obscuring the fingerprintable attributes of the VoIP app-specific STUN to confirm whether the differentiated behavior is limited to the fingerprintable VoIP app STUN.

The detection module analyzes and classifies responses to probes issued by the scanner module in order to identify potential censorship behavior. Probe outcomes fall into three categories: UDP responses, ICMP responses, or no response. We define identification criteria for each possible pairing of probe type and observed response (see Section 4.2.3). Our key insight is to detect anomalies where responses to vanilla STUN probes differ from those elicited by VoIP app–specific STUN probes, and to subsequently validate such anomalies using modified app-specific probes. If app-specific probes trigger responses that differ from vanilla probes, while the modified-app probes elicit responses indistinguishable from vanilla probes, we infer discriminatory treatment of the VoIP application, and thus confirm censorship.

We used *Revelio* to analyze a carefully selected set of 71 countries (including known censors and non-censor countries) and 11 popular VoIP apps. *Revelio* identified 19 countries (including Asian, Middle Eastern, and African countries, as well as one in Europe) that engage in VoIP filtering across 5 VoIP apps (WhatsApp, Telegram, Messenger, Signal, and Instagram). We further reverse-engineer the censorship infrastructure and uncover 4 middlebox types, each with multiple subtypes, specialized to block VoIP calls. The identified middleboxes employ a range of techniques: some simply drop VoIP app packets, while others reply with spoofed ICMP unreachable responses. We even found middleboxes that throttle VoIP app traffic, including random dropping and dropping in an arithmetic progression (see Section 5.3.3 for details).

We additionally analyze the blocking of domains associated with VoIP applications in the target countries to understand the layers at which VoIP censorship is enforced. To this end, we leverage Hyperquack [55], a remote measurement technique, and conduct over 7.56 million measurements across target regions. This analysis uncovers distinct censorship strategies: in some countries, VoIP applications are blocked at both the domain and STUN levels, while in others, application domains are unhindered but STUN traffic is selectively filtered. The latter results in partial functionality, where text-based messaging succeeds but VoIP calls consistently fail, indicating a more granular and application-aware form of censorship (see Section 5.4 for details).

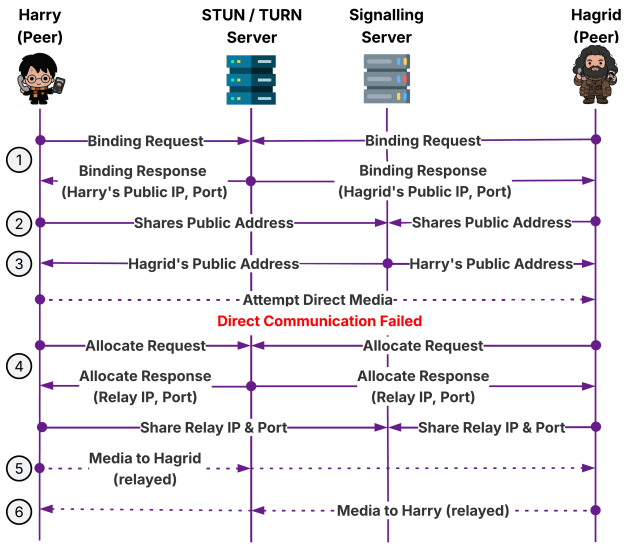


Figure 1: STUN Protocol: Peers attempt direct communication using Binding packets. If direct communication fails, they use Allocate packets to establish communication using a common STUN/TURN server.

Finally, we conduct a fingerprintability analysis of popular VoIP apps from a censor's perspective and find that the most widely used apps expose multiple stable identifiable features, making them straightforward to block. Our analysis shows that effective blocking does not require deep protocol understanding or complex traffic analysis; instead, it can rely on a small number of easily observable features. These findings underscore the need for a broader discussion about the responsibility of VoIP application developers and technology providers, more generally, to prioritize *protocol resilience*. In many cases, the distinctive features leveraged for censorship can be removed or obfuscated with minimal impact on app functionality, yet remain unaddressed in current designs.

2 Background

Modern VoIP applications such as WhatsApp, Telegram, and Signal rely on direct or relayed peer-to-peer communication to support low-latency voice and video calls. However, most users access the Internet from behind NATs and firewalls, which prevent direct inbound connectivity. To establish calls, VoIP applications therefore rely on the Session Traversal Utilities for NAT (STUN) protocol [42, 52] and its extension, TURN [32], to discover publicly reachable IP/port mappings or to allocate relay resources when direct connectivity fails.

STUN/TURN exchange occurs during the early stages of VoIP call establishment, before media transmission begins. Typically, users first exchange coordination messages over en-

crypted TCP/TLS connections maintained by the application backend. The application then sends STUN/TURN requests to dedicated servers, often operated by the VoIP provider itself, to perform NAT traversal and/or relay allocation. Once connectivity is established, voice or video traffic is transmitted either directly to the peer or indirectly via relay channels over UDP. Consequently, disrupting the STUN/TURN exchange prevents successful VoIP calls while leaving other application features, such as messaging and media sharing, unaffected.

In the basic workflow (see Figure 1), ① a client sends a *Binding Request* to a publicly reachable STUN server, which replies with a Binding Response containing the client's externally visible transport address. ② Peers exchange this information via a signaling server. ③ Peers use this information to attempt direct communication. When direct paths are unavailable, e.g., due to restrictive firewalls, applications fall back to TURN. ④, ⑤, ⑥ TURN introduces additional methods, most notably *Allocate*, which requests the server to reserve a public relay address and port on behalf of the client, enabling media traffic to be relayed through the server.

Importantly, TURN messages reuse the STUN message format; thus, an Allocate request is a TURN method carried in a STUN-formatted packet. Both Binding and Allocate messages are typically exchanged over UDP to minimize latency during call setup. A STUN/TURN message consists of a fixed 20-byte header containing the message type, length, a magic cookie (`0x2112A442`), and a transaction identifier, followed by optional attributes. Apps frequently include proprietary and application-specific attributes, causing STUN/TURN traffic to exhibit distinct fingerprints.

From a censorship perspective, STUN and TURN traffic provide an attractive point of control because these messages are usually unencrypted, occur early in session establishment, and are required for both direct and relayed VoIP communication. Lipphardt et al. [41] demonstrate that selectively dropping STUN exchange (Allocate) can prevent NAT traversal and relay allocation, disabling voice and video calls.

3 Related Work

A substantial body of prior work has examined Internet censorship across a wide range of national contexts. Early and subsequent measurement studies have documented censorship mechanisms and policies in countries such as Cuba [1], Syria [15], Iran [8], Saudi Arabia [4], Greece [63], Italy [2], Pakistan [13, 44], India [68], Russia [50], Spain [62], and Turkmenistan [47], among others. Collectively, these studies reveal a diverse set of censorship techniques, including DNS manipulation, IP blocking, TCP reset injection, SNI-based filtering, and application-layer interference.

A particularly large fraction of the literature has focused on China, driven by the technical sophistication, scale, and continuous evolution of the Great Firewall [67]. Prior work has analysed its DNS and IP-based blocking [35], collateral damage and overblocking effects [39], active probing and protocol fingerprinting [22, 36], keyword-based filtering [64], and censorship dynamics over time [7, 16, 19, 43]. More recent studies further examine adaptive censorship strategies, regional heterogeneity, and the interaction between censorship and circumvention systems, highlighting the Great Firewall as both a technically complex and rapidly evolving system. For instance, Zohaib et al. [70] analyse how the Great Firewall of China has begun decrypting and inspecting QUIC Initial packets to perform SNI-based censorship of QUIC traffic to specific domains, revealing a distinct blocklist and heuristic filtering rules. Wu et al. [66] recently identified and characterised an emerging regional censorship system in Henan Province, China, showing that provincial middleboxes independently implement HTTP Host and TLS SNI filtering that can block millions of domains, sometimes more aggressively and with different injection behaviours than the national Great Firewall. Tai et al. [56] took a different direction; they developed IRBlock [56], a multi-protocol measurement system that regularly probes the Great Firewall of Iran (GFI) to characterise DNS poisoning, HTTP/HTTPS blocking, and UDP traffic disruption across Iran, uncovering millions of censored domains and IPs and extensive collateral overblocking.

Complementary to transport-layer measurements, Niere et al. [46] study the deployment of Encrypted Client Hello (ECH) as a censorship-circumvention mechanism and evaluate its practical effectiveness. They demonstrate that while ECH encrypts sensitive TLS handshake metadata, the benefits of circumvention remain limited due to low server adoption, blocking of ECH-enabled endpoints, and interference with encrypted DNS by the censor.

Beyond country-specific analyses, several large-scale longitudinal measurement platforms have been developed to systematically monitor censorship events worldwide. Projects such as OONI [48], ICLab [45], Quack [61], Hyperquack [55], and Censored Planet [54] provide continuous, global visibility into web censorship practices. However, none of these platforms directly measure VoIP censorship. Notably, OONI has dedicated web censorship tests for popular messaging and VoIP apps such as WhatsApp, Signal, Telegram, and Messenger. It also includes an experimental STUN reachability test that checks whether STUN servers are reachable on a network by sending STUN binding requests and monitoring responses. However, our analysis shows that VoIP filtering primarily targets app-specific STUN allocate requests rather than generic STUN binding requests, a behavior that OONI's tests do not capture.

Note that an initial work on VoIP censorship was conducted by Lipphardt et al. [41], who studied the VoIP censorship mechanics in Saudi Arabia and the UAE. One of the authors traveled to the countries to conduct measurement tests because they were unable to remotely trigger censorship. Thus, in this work, we present *Revelio*, a platform for remotely and comprehensively studying global VoIP filtering.

4 System Design

4.1 Measurements Infrastructure

To investigate censorship practices for VoIP applications worldwide, it is essential to obtain vantage points within the target regions. Prior studies generally adopt one of two approaches to acquire such vantage points. The first involves acquiring locally hosted infrastructure, either by renting regional VPNs or VPSs [45], or by recruiting in-country volunteers to originate measurement probes [26]. The second approach relies on remote measurements, in which probes are sent from external machines toward carefully selected vantage points within the network under test [54]; these probes, or their resulting responses, then reveal censorship mechanisms.

Both approaches entail distinct trade-offs. Locally deployed servers enable more detailed, fine-grained analysis but introduce significant logistical challenges, including acquiring regional VPSs or VPNs and recruiting in-country volunteers. Furthermore, traffic originating from local VPNs or VPSs may not reliably trigger censorship mechanisms [26, 45, 54]. Volunteer-based measurements also raise ethical and safety concerns, as authorities in some countries have taken legal action against individuals accessing blocked resources [59]. In contrast, remote measurements facilitate large-scale studies without relying on volunteers. Their effectiveness, however, depends on whether probes originating outside the region under test can trigger censorship mechanisms.

In this work, we rely on remote measurements to identify STUN-based censorship across 71 countries, as this approach enables comprehensive, large-scale measurement campaigns.

4.1.1 Finding Vantage Points

To conduct remote measurements, we must first identify endpoints that can be used to perform censorship tests. Because this study focuses on VoIP call blocking, STUN servers constitute the most relevant measurement endpoints (as detailed in Section 2). Accordingly, our first step is to identify public STUN servers in each country under test. Since STUN protocol operates on UDP port 3478, we use `Zmap` [20], a widely used tool, to perform Internet wide port scan and identify hosts with an open STUN port. We obtain country-level IP prefixes from the BGP Report [21] and the CIDR dataset [24].

However, consistent with the findings of Lipphardt *et al.* [41], our initial scans reveal negligible publicly reachable STUN servers, *e.g.,* in the UAE, we obtained 2-3 IPs that have port 3478 open. We observed similar low counts in other countries, with many of them being 0. Lipphardt *et al.* also explored using VPNs in the target country but were unable to trigger censorship, potentially because such endpoints are not subject to filtering. We also observed no VoIP filtering when making calls via VPN endpoints in the same target countries (i.e., the UAE and Saudi Arabia).

To ensure sufficient endpoints for our measurements, we explore two complementary approaches. The first approach explores increasing the set of responsive IPs, whereas the second aims to utilize even the remaining unresponsive IPs.

In the *first approach*, we examine and address potential limitations in the default `Zmap` scanning methodology. To investigate this issue, we deployed multiple variants of vanilla STUN servers on cloud infrastructure and attempted to detect them using the default `Zmap` scanning methodology on UDP port 3478. We used the widely popular STUN server implementation `Coturn` (versions 4.5.2, 4.6.x, 4.7, and 4.8) in its default configuration. Surprisingly, `Zmap` failed to identify even our own deployed STUN servers. Further investigation revealed that STUN servers do not respond to generic UDP probes, even when the port is open [52]; thus, they remain invisible to generic `Zmap` UDP port scans, which rely on an explicit response to mark UDP port 3478 on an IP as open.

To get past this limitation, we examined the RFC-compliant behavior of STUN servers and observed that implementations should also listen on TCP port 3478, primarily as a fallback mechanism. All of our self-hosted STUN implementations similarly accept connections on TCP port 3478, and a `Zmap` scan over TCP successfully identified them as live. Motivated by this observation, we repeated our global scans over TCP for a subset of countries and observed a substantial increase in the number of discovered STUN servers, e.g., for the UAE, we get 2947 IPs that are open on TCP port 3478, which is more than 982 times than IPs open on port UDP 3478 (3 IPs).

Although we observed a substantial number of servers listening on the TCP port, one cannot conclude that these servers are STUN servers, as another non-STUN service could be running on that port. For our study, we needed reliable, confirmed STUN servers. Thus, we use `Zmap` to send a crafted, vanilla STUN `ALLOCATE` request to each candidate server and record the resulting responses. Receipt of vanilla STUN responses, i.e., a valid `ALLOCATE SUCCESS` or a valid STUN `ALLOCATE ERROR`, confirms that the server is functional and running a STUN service.

In contrast, the absence of a response or the receipt of network-layer errors (e.g., ICMP destination-unreachable) indicates that the IP address is non-functional or non-responsive on UDP port 3478. Note that while `Zmap` can be used to send STUN requests, it currently does not have a mechanism to parse the responses as STUN `ALLOCATE SUCCESS` or STUN `ALLOCATE ERROR`. To this end, we modify the `Zmap` source code to support parsing STUN responses (see Section 5.1).

The *second approach* to expanding the set of usable vantage points is to explicitly incorporate otherwise non-responsive IP address space into our censorship-detection methodology. The key idea is to send packets to non-responsive addresses that trigger censorship and analyze the resulting response. In the absence of censorship, probes directed at non-responsive end hosts typically elicit no response or a consistent network-level error. In contrast, if a censorship middlebox is triggered

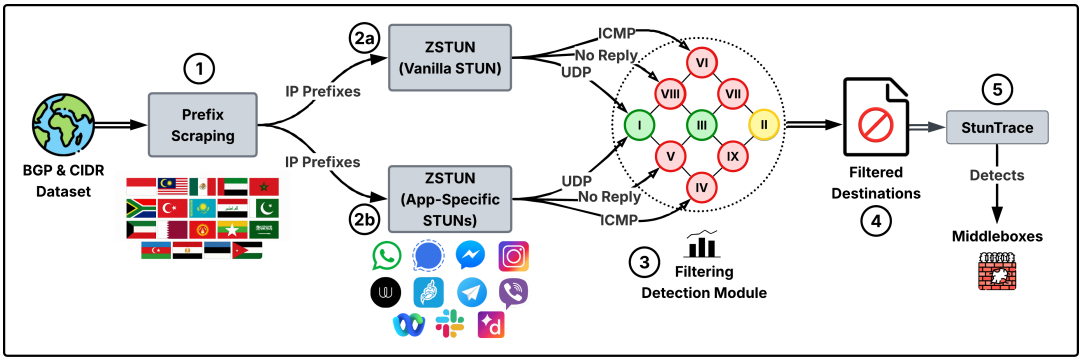


Figure 2: Overview of **Revelio**. **(1)** Collects country-level ASNs and IP prefixes. **(2a)** Establishes a baseline by probing the IPv4 space with vanilla STUN packets, classifying responses as UDP (STUN), ICMP error, or No response; **(2b)** repeating the scan using app-specific STUN packets. **(3)** The filtering detection module compares the 9 (3×3) response combinations to identify censorship (e.g., STUN response for vanilla packets but no reply for app-specific packets indicates filtering, Case V). **(4)** Filtered destination IPs are stored in a database. **(5)** STUN traceroute is used to infer the type of middlebox involved.

along the path, it may generate a distinct response, enabling us to infer the presence of filtering.

Overall, our methodology consists of scanning the entire IP address space of each target country using a modified version of `Zmap` (details in Section 5.1) that sends 10 vanilla STUN `ALLOCATE` requests per IP. We selected 10 probes because Lipphardt et al. [41] found that middleboxes typically trigger after 2–3 STUN requests. Given our remote study, we introduced redundancy ($\approx 3\times$). Increasing redundancy excessively would incur significant scan overhead. Thus, 10-packet balance detection reliability and responsible scanning.

This process yields two disjoint sets of IP addresses: *responsive and non-responsive*. As described in the next section, we develop separate detection techniques tailored to each set to identify evidence of filtering. An overview of our end-to-end measurement pipeline is shown in Figure 2.

4.2 Censorship Detection

Having obtained a set of usable vantage points, we next devise detection approaches for identifying STUN-based blocking. Because our goal is to detect blocking of popular VoIP applications, our methodology consists of three key steps: (1) extracting application-specific STUN packets, (2) sending these packets as probes toward networks in the target countries, and (3) analyzing the resulting responses to identify and characterize blocking behavior.

4.2.1 STUN Fingerprint Generation

To generate app-specific fingerprints, we executed each app and initiated its voice and video calling functionality. During each call, we captured all network traffic and isolated the corresponding STUN requests. We repeated this experiment five times per app to ensure consistency and recorded the resulting app-specific STUN messages. To check whether an app has a STUN fingerprint, we compared the captured packets against vanilla STUN messages defined in the RFC. We labeled an app as having a unique STUN fingerprint if its packets deviated from the vanilla STUN format in any way and were not present in other apps. We later show in Section 5 that these features are useful for confirming and attributing STUN-based filtering.

4.2.2 Sending Censorship Triggering Probes

After extracting app-specific fingerprints, we dispatched measurement probes across the entire IP address space of each country under test. For each endpoint, we transmitted three classes of probes: (1) a standards-compliant STUN packet (**P1-vanilla**), (2) an app-specific STUN packet containing fingerprintable attributes (**P2-app**), and (3) a modified version of the app-specific STUN packet with obscured fingerprints (**P3-modapp**). We recorded all corresponding responses for subsequent analysis.

4.2.3 Detection Logic

For each probe sent, we observe one of three possible outcomes: (1) a vanilla STUN `ALLOCATE` response (**R1-vanilla**), (2) an ICMP error (**R2-ICMP**), or (3) no response (**R3-Noresp**).[2] We assign a response category only when at least 8 out of 10 probes yield the same outcome. This value was selected based on experiments from (1) our own STUN servers in Bangalore, New York, and Singapore, where we consistently received responses for at least 9 of 10 probes, and (2) from publicly reachable STUN servers (e.g., Google), where we always received at least 9 responses.

Considering the first two probe types (**P1-vanilla** and **P2-app**) and the three response categories (**R1-vanilla**, **R2-**

[2]Because STUN operates over UDP, there are no protocol-level indicators of blocking analogous to TCP *RST* packets.

Case	P1 (vanilla)	P2 (App)
I	R1-vanilla	R1-vanilla
II	R2-ICMP	R2-ICMP
III	R3-Noresp	R3-Noresp
IV	R1-vanilla	R2-ICMP
V	R1-vanilla	R3-Noresp
VI	R2-ICMP	R1-vanilla
VII	R2-ICMP	R3-Noresp
VIII	R3-Noresp	R1-vanilla
IX	R3-Noresp	R2-ICMP

Table 1: Probe response cases: green marks non-anomalous, red marks censorship, and yellow marks non-anomalous cases with potential censorship.

ICMP, and **R3-Noresp**), we obtain a total of nine possible probe–response combinations (see Table 1). We classify and analyze each case individually below.

Among the nine possible probe–response combinations, Cases I, II, and III represent non-anomalous behavior and therefore should be classified as unfiltered. Cases VI and VIII, while technically feasible, do not typically represent middlebox filtering because a vanilla response to app-specific STUN does not indicate such filtering.

The remaining four cases indicate potential filtering. Specifically, Cases IV and V correspond to scenarios in which vanilla STUN probes elicit a normal response, while application-specific STUN probes result in either an ICMP error or no response. This asymmetry indicates discriminatory treatment of application-specific STUN traffic. Similarly, Case VII is characterized by an ICMP error in response to vanilla STUN probes and no response to application-specific STUN probes. Interestingly, Case IX also suggests potential filtering, as it exhibits no response to vanilla STUN probes but an ICMP error for application-specific STUN probes. Unlike the other three filtering cases, which rely on responsive IP addresses, Case IX leverages otherwise non-responsive IP address space to infer filtering behavior.

To validate these potential filtering cases, we conduct additional *confirmation tests* using modified application-specific probes (**P3-modapp**). The purpose of these probes is to obscure uniquely identifiable fingerprints of the app-specific STUN packets and observe whether the responses change accordingly. For example, in the case of WhatsApp, we alter the values of application-specific unknown attribute headers, specifically `0x4000` and `0x4024`, which uniquely identify its STUN implementation. While multiple modifications are possible, including removing unknown attributes altogether, we ensure that such changes do not transform the packet into a vanilla STUN message. We list down the fingerprintable attributes in Table 4.

If the responses to the vanilla probe (**P1-vanilla**) and the modified app-specific probe (**P3-modapp**) are identical but differ from those of the app-specific probe (**P2-app**), we classify the behavior as confirmed filtering.

The case of ICMP responses: For much of the discussion above, we have treated ICMP responses as homogeneous. However, there are multiple types and subtypes of ICMP responses. In our approach, we treat each ICMP response type differently, which is why Case II may still result in censorship behavior if the received ICMP responses for **P1-vanilla** and **P2-app** are of a different type.

The case of throttling: Our existing approach detects anomalous request-response pairs across different probe types. However, detecting throttling as a censorship mechanism requires additional steps. As previously mentioned, we consider only the results from probes that return the same response 8 out of 10 times for a given target IP. We then investigate the remaining probes (with fewer than 8/10 responses) as potential cases of throttling. We perform two additional steps to distinguish throttling from transient network behavior. First, we repeatedly send 10 **P2-app** probes five times and record whether fewer than 8 responses are consistently observed, ruling out transient network behavior. If the conditions hold, as a confirmatory next step, we send 10 **P3-modapp** probes. We classify the case as throttling if we receive more than 8 out of 10 responses for **P3-modapp** probes.

The case of STUN server poisoning: While censorship is typically performed by middleboxes along the path, we cannot rule out STUN poisoning as a filtering mechanism. This is similar to DNS poisoning, in which STUN servers themselves implement censorship by returning vanilla STUN responses containing fake or unusable information. The cases VI and VIII may thus indicate censorship by STUN poisoning and are thus marked accordingly in Table 1.

Blocking at the domain level: While blocking STUN is an effective way to censor VoIP apps, disrupting communication at the HTTP/HTTPS level is another way to block them. Thus, we conducted additional experiments to understand the blocking status of domains associated with each app. To this end, we used the remote measurement technique *Hyperquack* [54] to detect interference at the HTTP and HTTPS levels. Hyperquack operates by generating and transmitting HTTP(S) requests to publicly reachable web servers within the target network, and then analyze the responses to detect interference introduced by network intermediaries, such as middleboxes.

4.3 Characterizing Censorship Infrastructure

The next stage of our system design focuses on characterizing the middleboxes and censorship infrastructure responsible for the identified filtering behavior. In this analysis, our goal is to determine the location of the middleboxes along the network path, their IP addresses, and their functional type.

To infer middlebox placement, we use standard TTL-based techniques employed by tools like Centrace [49] (and its predecessors [67, 68]). However, Centrace is designed for TCP-

based measurements and targets HTTP and TLS interception; therefore, it cannot be directly applied to our setting, which involves UDP-based STUN traffic.

To address this limitation, we develop `StunTrace`, a custom tool inspired by the same TTL-based principles tailored specifically for STUN. The key idea is to send censorship-triggering packets with increasing TTL values and observe the intermediate hop from which the responses are received. `StunTrace` sends user-specified sequences of STUN probes, including any combination of **P1-vanilla**, **P2-app**, and **P3-modapp** packets, with incrementally increasing TTL values. By observing where along the path censorship is encountered, we confirm the presence of STUN-based middleboxes and attempt to identify the AS hosting them (see Table 3).

To run `StunTrace`, we selected the IP addresses along whose network paths we observed censored responses during our scans. Since running traceroutes for every target IP (in millions) is impractical, we randomly selected one IP address from each /24 prefix. This approach ensured a uniform distribution of targets for `StunTrace` while significantly reducing the measurement overhead and completion time.

After identifying candidate middleboxes, we further reverse-engineer their behavior by determining the exact packet sequences that trigger censorship and grouping the middleboxes based on these conditions. Throughout this process, we also document protocol and implementation-specific idiosyncrasies exhibited by the middleboxes.

This analysis is particularly valuable because, while there is extensive prior work characterizing middleboxes and censorship mechanisms for protocols such as DNS, HTTP, and HTTPS [10, 53, 61], there is little existing understanding of the types of middleboxes and techniques used to disrupt VoIP communication through STUN filtering.

5 Measurements and Results

5.1 Implementation Details

Our measurements were conducted from a single measurement machine located in Amsterdam. The machine was provisioned with 8GB RAM and was running Ubuntu 22.04 (LTS) x64. Furthermore, since `Zstun` scans require high bandwidth to complete within a given duration (a day in our case) we provisioned 300 Mbps of bandwidth. For the scans, we use `Zstun`, which is built on `Zmap` version 4.3.4. We also validated our findings from multiple geographic locations: Bangalore, Sydney, Singapore, and San Francisco. For these experiments, we ran *Revelio* only for the 19 countries where filtering had previously been identified from the Amsterdam measurement machine. Across all locations, we observed the same STUN filtering as observed from the Amsterdam.

Our measurements were conducted over five months, from August 2025 to December 2025. The first four months were devoted to pilot studies to identify issues and refine the methodology. These efforts involved extending ZMap to support STUN scanning (`Zstun`, see Section B), developing insights from P3 probes, testing and integrating `StunTrace`, and detecting spoofed ICMP error messages. We began our research by using responsive STUN servers as our destinations. However, during the pilot studies, we enhanced our methodology to use the country's non-responsive IP space and developed the cases I–IX in Table 1. We eventually launched the measurement campaign from 1st December 2025 to 30th December 2025, conducting scans twice daily. All results reported in the paper are based on these 1-month measurements.

5.2 Target Countries and Apps

To conduct our study, we included all countries classified as *Not Free (21)* or *Partly Free (28)* by the "Freedom on the Net" ranking [31]. We also included a few additional countries that are likely to engage in censorship but are not included in the Freedom House Index. They are Israel [28], Kuwait [51], Oman [12], Cyprus [18], Palestine [38], Qatar [58], Yemen [33], Turkmenistan [47], Nepal [17], Afghanistan [30], and Syria [15]. All of these countries either have had recent political and/or war-like situations or have been in such a state for some time. We also include a set of "free" countries as a control group to validate *Revelio*'s detection methodology. Since large-scale UDP measurements can exhibit packet loss, routing anomalies, or transient failures, measuring only suspected censoring countries could lead to false positives. By comparing results against countries generally considered uncensored, we verify that the observed STUN anomalies, such as selective dropping or ICMP spoofing, are indicative of deliberate VoIP censorship rather than of normal network behavior. We randomly added the following free countries from Freedom House [31]: the United Kingdom, Germany, Argentina, South Africa, Taiwan, Chile, Estonia, Serbia, Georgia, Armenia, and Costa Rica. This resulted in a total of 71 countries.

Since there is no list of VoIP apps specifically designed for testing VoIP filtering, we compiled a list of popular VoIP apps from blogs and online articles [11, 29]. We compiled a list of 31 apps and confirmed that they were popularly available on the Google Play Store (as Google Play is potentially popular in regions where such filtering is likely to be observed). Among these, we were able to initiate VoIP calls from 19 apps and included them in our evaluation. Note that for 11 such apps (WhatsApp, Telegram, Viber, Signal, Messenger, Instagram, Slack, Dialpad, Webex, Jitsi, and Wire), we were able to extract fingerprintable STUN packets. The unique fingerprints for each app are represented in Appendix Table 4.

5.3 Results

We organize our findings into three parts: (i) evaluation of vantage point stability and suitability, (ii) identification of

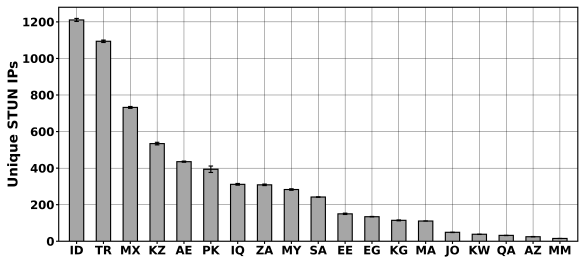


Figure 3: Total STUN servers in each country across scans.

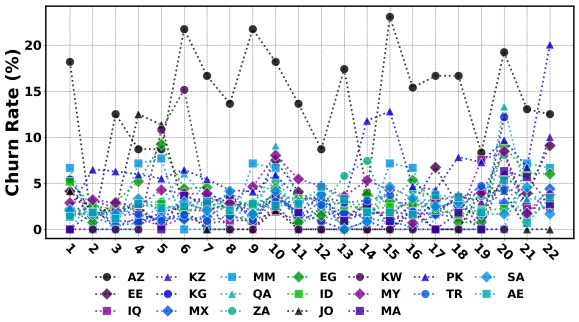


Figure 4: STUN servers churn across scan iterations.

censorship mechanisms across countries, and (iii) additional insights into the diversity and operational behavior of the discovered middleboxes.

5.3.1 Characterization of Vantage Points

Using `Zstun`, we discovered STUN servers in all 71 countries included in our study. However, evidence of filtering was observed in only 19 of these countries; therefore, we focused on STUN servers in these 19 countries. Figure 3 presents the total number of STUN servers discovered in each country. Indonesia (ID) hosts the largest number of open and responsive STUN servers (about 1,200), closely followed by Turkey (TR). In contrast, Myanmar (MM) and Azerbaijan (AZ) lie at the lower end of the spectrum, with 15 and 24 STUN servers identified, respectively.

Next, we evaluated STUN server stability by measuring churn across repeated scans. Churn is calculated as the ratio of STUN servers added or removed in the last scan to the total number of servers found in the last scan. Figure 4 presents churn measurements across all 19 countries. Most countries exhibit very low churn rates (<5%), indicating stable vantage points. Five countries, Azerbaijan (AZ), Kuwait (KW), Qatar (QA), Kyrgyzstan (KG), and Jordan (JO), show churn exceeding 10% in some scans. Overall, these results indicate that the discovered STUN servers remain largely stable over time.

5.3.2 Core Findings

We now present the core results of filtering observed across the 19 countries. As described in Section 4.2.3, filtering evidence is inferred from the probes sent (P1–P3), and the corresponding responses received (R1–R3). Several request–response patterns indicate filtering behavior (Cases IV, V, VI, VII, VIII, and IX). Note that we found no evidence of STUN poisoning; therefore, cases VI and VIII did not occur. Using `Zstun`, we scan the entire IPv4 address space for a given country with 10 probe packets/IP address. We send multiple probe packets to mitigate the effect of random packet drops and other confounding factors. We adhered to the best ethical practices while conducting these large-scale measurements (see Section 7).

UDP Censorship: Cases IV and V fall under this category of censorship. Figure 5 shows Case IV, where, on sending vanilla STUN allocate (P1) probes, we receive vanilla STUN responses (R1), while app-specific packets (P2) trigger an ICMP error (R2) or throttled responses. The bar plot depicts the number of IPs that responded to different types of packets. For instance, the first bar shows that 409 IPs responded with more than 8 UDP (STUN Allocate/Error) responses when probed with 10 vanilla STUN Allocate packets (P1). We consider such IPs to be a stable set of STUN servers as they responded to the majority of the probe packets. Since we send unsolicited STUN packets, we expect a STUN allocate error from the servers. In some cases, improperly configured STUN servers also respond with vanilla STUN reply. Nevertheless, if we receive any STUN packet in response, we classify the IP as a STUN server.

The second bar shows the result of sending WhatsApp-specific STUN packets (P2). Compared to the first bar, we observe a clear decline in the number of IPs that responded with STUN packets (275 IPs; indicated by a downward red arrow). Instead, we observe two new patterns, IPs (131) that previously responded with ≥ 8 STUN responses: (1) some of them responded with < 8 STUN responses (in light green), (2) some responded with ≥ 8 ICMP error messages (in blue). (1) indicates the presence of a throttler, and (2) suggests that the middlebox is responding with ICMP error messages spoofing the destination IP.

We confirm our hypothesis by changing the WhatsApp-specific STUN attributes (`0x4000` -> `0x4001` and `0x4024` -> `0x4025`) and sending the modified WhatsApp STUN packet (P3), following the same procedure as for P1 and P2. In the third bar, we see a clear increase in the number of IPs that responded with vanilla STUN packets (indicated by the upward black arrow). The responding IP count is very close to the first bar, where we sent P1 packets. This strongly indicates that our probes encountered attribute-specific filtering.

We conducted the same scans twice daily in December 2025 and observed a similar trend of a drop in P2 probes, with only minor variation in the volume of filtered IPs between successive scans (relative difference within the range of 0–

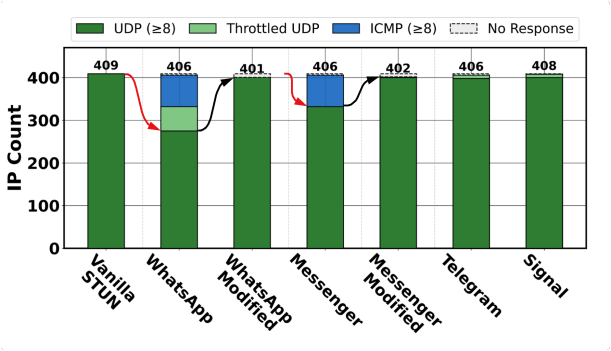


Figure 5: Anomaly in UDP responses: Vanilla STUN returns UDP, but a sizable portion of app-specific STUN returns ICMP for WhatsApp and Messenger in the UAE (Case IV).

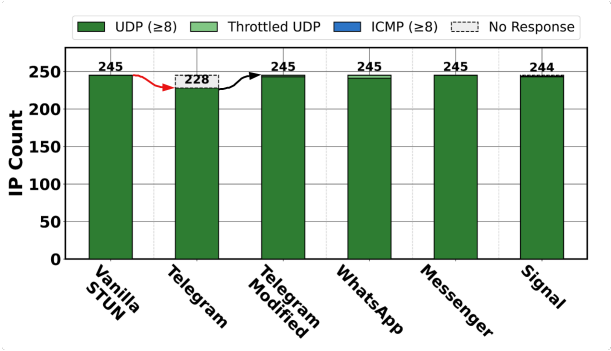


Figure 6: Anomaly in UDP responses: Vanilla STUN returns UDP, but a sizable portion of app-specific STUN sends No Response for Telegram in Saudi Arabia (Case V).

12.35%). Among multiple scans, we take the intersection of such destination IPs that respond differently to P1 and P2, but the same to P1 and P3. We call such a final set of IPs a *filtered (destination) IPs*. We explain in Section 5.3.3 that we use these IPs to characterize the middlebox's behavior that causes filtering.

We observe the same interference pattern for Facebook Messenger STUN packets that carry `turner.facebook` in the `Realm` attribute (the fourth bar). Simply by changing the attribute to a random string, we bypassed the middlebox and received the expected STUN responses from the destination servers (fifth bar). We observed no such VoIP filtering for Signal and Telegram. Overall, we observe this filtering pattern consistently across all 19 countries. We show in Table 2 what apps are blocked in what countries.

Similarly, Figure 6 represents Case V, where in Saudi Arabia, vanilla STUN generates a UDP response, but Telegram-specific probes resulted in a sizable portion of IPs not responding at all. We can observe from the figure that the number of IPs in the first bar falls from 245 to 228 in the second bar. Although only a small proportion of 18 IPs exhibit this behavior, sending modified Telegram probes that obscure the `realm` field restores the number of UDP responses to 245, confirming censorship behavior. From our scans in December 2025, the relative difference between successive scans remained within the range of 0–10.53%. We observed this type of censorship in all 19 countries for at least one app.

The results in Figure 5 and Figure 6 naturally raise the question of why the observed proportion of dropped probes is relatively small. Our confirmatory probes (P3), together with a subsequent analysis identifying the responsible middleboxes, verify that these cases do indeed correspond to intentional filtering. The limited fraction of affected probes, therefore, likely reflects partial path coverage for the deployed STUN servers, rather than weak filtering. In particular, it suggests that censorship infrastructure is not uniformly deployed across all network paths and that the relevant middleboxes are positioned so that only a subset of routes to the target IP address space traverse them. As a result, filtering is observable but does not affect most paths. Thus, we devised additional heuristics to detect filtering. We now describe how we leveraged each country's remaining IPv4 address space to identify filtering, including cases where destination IPs returned ICMP error messages or produced no response.

No response Censorship: Figure 7 represents Case IX, where vanilla STUN (P1) resulted in No response (R3), but app-specific STUN (P2) resulted in ICMP errors (R2).

In the figure, the top dotted black horizontal line represents the number of IPs that did not respond to our P1 probes at all (recall, we sent 10 packets/IP). We call them *unresponsive IPs*. We sent P2 and P3 packets to these unresponsive IPs. Ideally, we should not receive any responses to these probe packets either. However, for WhatsApp and Messenger STUN packets (first and third bars), ICMP errors were received from IPs that did not respond to vanilla STUN at all. However, once modified app-specific STUN packets were sent (P3), the number of destination IPs that responded with ICMP errors declined sharply (second and fourth bar). A significant fraction of IPs (99.83%), did not respond to the modified packets at all. This strongly indicates that the middlebox responds with ICMP Destination Unreachable messages, spoofing the destination IP address, when it encounters blocked STUN packets. The scale of responses was also considerable: about 1.5M IPs responded with an ICMP error for P2, and only a few thousand responded for P3. We found this kind of filtering behavior only in the UAE.

ICMP based Censorship: We now study cases in which P1 probes elicit ICMP responses while responses to P2 probes vary (cases VII and II). The first result is shown in Figure 8 for Myanmar. In this result, we observed ICMP errors for P1, whereas we received no response for P2, thereby representing case VII. From the figure, we can clearly see that the first bar shows ICMP errors for 4593 IPs. However, the second bar representing Messenger resulted in no response in over 90%

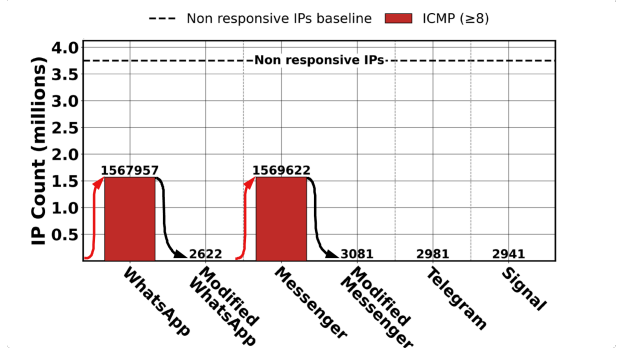


Figure 7: Anomaly in No responses: Vanilla STUN returns No Response (dotted line), but app-specific STUN returns ICMP for WhatsApp and Messenger in the UAE (Case IX).

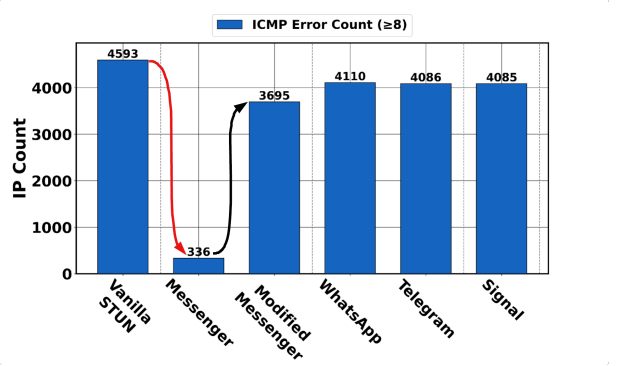


Figure 8: Anomaly in ICMP responses: Vanilla STUN returns ICMP errors, but the majority of app-specific STUN gives no response for Messenger STUNs in Myanmar (Case VII).

of cases, indicating anomalous behavior toward Messenger. Obscuring the app-specific STUN fingerprint (P3 probes) and sending them results in the same ICMP errors as vanilla STUN packets, confirming censorship. We observed this kind of censorship in Myanmar, Jordan, Azerbaijan, and the UAE.

For this result, the specific type of ICMP error received for P1 is immaterial, since P2 consistently resulted in no response. However, in actual experiments, we received two types of ICMP error responses: an ICMP destination-unreachable message and a TTL-expired message. We now discuss cases where examining both the ICMP error responses individually also helped us identify filtering cases.

ICMP TTL Exceeded Message: The first case was the `ICMP TTL-expired` message in response to P1 probes and a mix of `ICMP destination unreachable` as well as TTL expired response to P2 probes. This case is shown in Figure 9, where vanilla STUN produced ICMP TTL errors from 15,325 IPs, whereas WhatsApp-specific STUN produced ICMP destination unreachable for 7744 IPs (more than 50% cases). However, when we sent P3 (modified WhatsApp STUN), we again received TTL-expired messages from most of the said IPs (14,286), confirming censorship behavior. We observed a similar trend for Messenger-specific STUN packets (P2). Note that this type of filtering represents Case II.

While this behavior was clearly discriminatory against specific VoIP apps, the interplay between censorship mechanisms and ICMP TTL-exceeded messages is a bit puzzling: why would packets with the default TTL value result in TTL-expired messages along paths to thousands of IP addresses in a country? To better understand the mechanics behind them, we conducted further investigation. First, we analyzed whether the TTL expiration behavior is specific to STUN packets or is protocol- and port-agnostic. To do this, we conducted additional scans of the initial set of IPs that responded to vanilla STUN packets with ICMP TTL-exceeded messages (15235, first bar in the Figure 9). We sent generic UDP and TCP packets on varying ports. Regardless of the protocol, we received an ICMP TTL-exceeded message for the target IPs. Second, we investigated whether the response is tied to the TTL value of our probes. To test this, we sent packets with TTL values up to 200 and still received the same TTL-expired response. Overall, these two observations confirm the presence of *routing loops* on these target IPs.[3]

However, why the VoIP app's packets were not affected by such loops remained unresolved. *We confirmed with the help of* `StunTrace` *that the middleboxes that were dropping packets were encountered before the routers that were causing routing loops.* We found this behavior in all 19 countries.

ICMP Destination Unreachable Message: The second category of ICMP responses we obtained was `ICMP destination unreachable`. We observed that P1 produced a relatively small set of IPs (23,585) that responded with these ICMP errors (see first bar in Figure 10). However, app-specific STUN packets (P2) resulted in a significant increase in the number of IPs responding with these ICMP errors (90,252; 3.83× increase). Again, when we sent P3, we observed a relatively low number of IP addresses responding with an ICMP destination-unreachable message (20640, third bar), a behavior consistent with the case when we sent P1. This again strongly indicates censorship.

On further investigation using `StunTrace`, we found that while the ICMP errors for P1 and P3 originated from the end hosts, those for P2 were injected by the middlebox by *spoofing* the end hosts' source IP addresses. This was a distinct behavior and explains why we observed fewer ICMP error responses for vanilla STUN packets, as these were generated by end hosts as an artifact of their implementation.

[3]We randomly selected 1000 IPs and observed that traceroute to these IPs always had repeated last two hops, confirming routing loops. A recent study by Alaraj et al. [3] also finds persistent routing loops on the path in about 0.6% of the IPv4 address space.

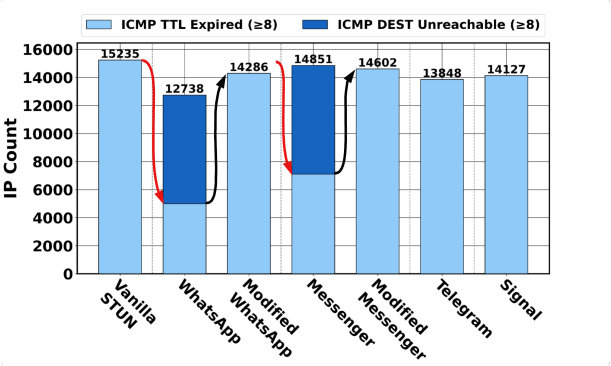


Figure 9: Anomaly in ICMP responses: Vanilla STUN returns ICMP TTL Expired, but a sizable portion of app-specific STUN returns ICMP destination unreachable, instead for WhatsApp and Messenger in the UAE (Case II).

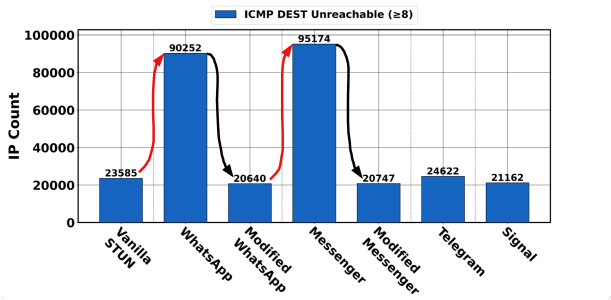


Figure 10: Anomaly in ICMP responses: Vanilla STUN elicits ICMP destination unreachable for a small proportion of IP space, but WhatsApp and Messenger app-specific STUN return considerably more (4×) such responses in the UAE. Note that the app-specific ICMPs were sent by the middlebox, but vanilla ICMPs were generated by end servers.

5.3.3 Type of Middleboxes

We used `StunTrace` to critically analyze the responses and characterize the mechanics and types of middleboxes. Our analysis identified *four categories and six subcategories*, the first report of such STUN-specific middleboxes (see Figure 11). We now describe and summarize the mechanics of the different types of middleboxes observed across countries in Table 2.

Type M1: Drop on first encounter When sending vanilla STUN packets (P1) with increasing TTLs (using `StunTrace`), we observe the expected traceroute path, with packets reaching the destination. However, when we send app-specific STUN packets (P2), in-path middleboxes drop them. To identify middleboxes, we select those destinations that respond to P1 but not to P2 (cases V and VII, as per Section 4.2.3). We classify these middleboxes into sub-categories:

Middleboxes reveal their IP addresses (**M1-A**): When we sent P1 probes using `StunTrace`, we observed a full traceroute path terminating at the destination. When we sent P2 probes, after a particular hop (short of the destination), we observed no ICMP TTL exceeded responses (`***`), indicating that the packets were dropped. However, on sending P3, we again see the complete path, confirming attribute-specific filtering.

Middlebox spoofs the source IP address (**M1-B**): These middleboxes exhibit two behaviors. In one variant, ICMP TTL-exceeded responses are generated for both vanilla and app-specific STUN packets. In the other, only vanilla STUN packets elicit TTL-exceeded responses, while app-specific packets receive none. Importantly, these TTL-exceeded responses are spoofed to appear as originating from the destination IP. We confirmed this using `StunTrace` with both P1 and P2 probes. This suggests non-standard filtering logic that inspects app-layer STUN fields. Even if the inspected packet contains no blocked patterns and the TTL has expired, the middlebox still generates a spoofed TTL-exceeded message.

Country	Categories	Applications
United Arab Emirates (AE)	M1-A,B,C, M3 M2-A,B,C, M4	WhatsApp, Instagram Messenger
Azerbaijan (AZ)	M1 (B)	Telegram
Estonia (EE)	M1 (A)	Telegram
Egypt (EG)	M1 (C)	Telegram
Indonesia (ID)	M1 (A, B)	Telegram
Iraq (IQ)	M1 (C)	Telegram
Jordan (JO)	M1 (A)	Telegram, WhatsApp
Kyrgyzstan (KG)	M1 (C)	Telegram
Kuwait (KW)	M1 (C)	Telegram
Kazakhstan (KZ)	M1 (C)	Telegram
Morocco (MA)	M1 (B, C)	Telegram
Myanmar (MM)	M1 (A, B)	Messenger
Mexico (MX)	M1 (B)	Telegram
Malaysia (MY)	M1 (B)	Telegram
Pakistan (PK)	M1-A, M4-A	Signal
Qatar (QA)	M1 (A)	Telegram
Saudi Arabia (SA)	M1 (A, C)	Telegram
Turkey (TR)	M1 (A)	Telegram
South Africa (ZA)	M1 (A)	Telegram

Table 2: Taxonomy of middlebox behaviors.

Middleboxes do not reveal their IP addresses (**M1-C**): These middleboxes behave similarly to IA; however, we cannot obtain their IP addresses because they do not respond with ICMP TTL-exceeded messages, even during a standard UDP traceroute. Nevertheless, we observe that sending blocked STUN packets consistently triggers filtering (drops the P2 probes), resulting in no responses.

Type M2: Drop after multiple encounters Similar to the M1 case, P1 probes elicited some form of response, either UDP or ICMP, whereas repeated P2 probes received responses only for the first 2–3 packets. This behavior indicates that the middlebox requires a threshold number of packets to trigger filtering. We confirmed this by sending multiple P3 probes, which never exhibited this behavior, consistent with observations reported by [41].[4] This behavior corresponds to cases V

[4]Please note that authors in [41] identified only this specific kind of filtering behavior. However, we report multiple types and sub-types of middleboxes.

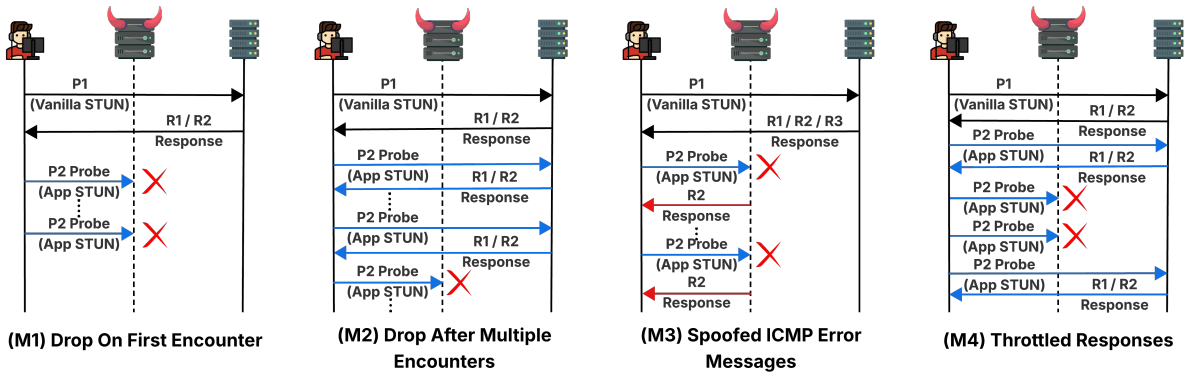


Figure 11: Types of STUN blocking middleboxes.

and VII described in Section 4.2.3.

Type M3: Middleboxes spoof ICMP Destination Unreachable Messages For vanilla STUN packets, we receive expected STUN responses; for blocked STUN packets, the middlebox spoofs the destination IP and sends an ICMP Destination Unreachable error response. Thus, they create a pretense for the client that the STUN server is unreachable. Depending on how the end destination responds, we detect these middleboxes in three ways. First, if the destination server is a legitimate STUN server, we receive R1 for P1 but ICMP Destination Unreachable for P2, confirming the censorship. (case IV) Second, if there is a routing loop along the path, P1 yields an ICMP TTL-exceeded message, but P2 again yields an ICMP destination-unreachable message, spoofing the packet's source IP (case II). Third, when the destination is unresponsive, P1 yields no response (as expected). But P2 again results in an ICMP destination-unreachable message, confirming the censorship (case IX).

Type M4: Throttled Responses Compared to P1 probes, the P2 probes receives fewer responses. Note that these are different middleboxes from the one in Type-II, because here we still receive responses, whereas in Type-II, responses stop entirely after 2-3 initial responses. Analyzing the response pattern, we categorized the middleboxes as follows:

Random Drops (**M4-A**): These middleboxes drop packets in a random pattern, resulting in traffic throttling.

Periodic Drops (**M4-B**): These middleboxes drop bursts of packets (e.g., 4–5 packets) followed by short intervals during which some packets (e.g., 1–2) are allowed to pass through. Such periodic breaks can effectively throttle the traffic.

Arithmetic Drops (**M4-C**): Similar to the previous category, these middleboxes periodically drop packets; however, the drop intervals follow an arithmetic progression. For example, if the first drop occurs at the 1st packet, subsequent drops typically occur at the 11th, 22nd, and 34th packets. Notably, the gap sequence periodically resets to its initial value. We confirm this behavior by sending 200 packets to the destination. The first wrap-around happened at the 34th packet. After the 34th, we observed the 44th, 55th, and 67th packets dropped. Wraparound happened again, and the pattern repeats.

On the statefulness of the middleboxes: In summary, our results suggest that the identified middleboxes are stateful. (1) Triggering doesn't occur when the fingerprintable attribute was modified while all the other packet headers remained identical. (2) We observed residual censorship (including M4B and M4C), where once triggered, middleboxes kept dropping packets for the triggering tuple for some duration (see Appendix Section C). (3) Sending uncensored probes (P1 & P3) first, followed by the censored probes (P2) for the same tuple value, does not result in blocking. This behavior suggests that the middlebox marked a tuple value as safe, which can later be used to send **P2** probes through it. Points (2) and (3) especially indicate state management, while (1) confirms no other feature was used for censoring.

Country	Filtered AS (%)	AS with Middlebox (%)	Filtered STUN (%)
Azerbaijan	36.67	1.11	5.56
Egypt	2.86	0.00	1.45
Estonia	7.98	0.00	34.29
Indonesia	1.99	0.04	7.02
Iraq	2.13	0.00	0.92
Jordan	25.00	4.17	1.92
Kazakhstan	2.42	0.00	0.90
Kuwait	1.39	0.00	7.89
Kyrgyzstan	3.45	0.00	1.35
Malaysia	4.65	0.39	7.82
Mexico	0.84	0.00	0.67
Morocco	7.14	3.57	1.82
Myanmar	39.68	0.79	8.70
Pakistan	13.38	1.34	29.90
Qatar	6.25	6.25	2.86
Saudi Arabia	3.70	0.62	4.63
South Africa	1.10	0.00	2.02
Turkey	2.43	0.00	2.62
UAE	36.23	2.90	15.18

Table 3: Filtered and Middlebox AS Statistics.

On AS-level Analysis: Using *Revelio* and `StunTrace`, we identify (1) the destination ASes along whose network paths we observed STUN filtering, (2) the ASes hosting the censoring middleboxes, and (3) the fraction of publicly reachable

STUN servers for which filtering was observed. Table 3 summarizes these findings. A value of 0 in the "AS with Middlebox" column indicates that a middlebox was detected, but its IP address was not revealed in the `StunTrace`, preventing us from identifying the precise AS hosting the middlebox.

Moreover, a high fraction of filtered ASes, such as Myanmar with nearly 40%, may indicate nationwide censorship deployment, whereas a much lower fraction, such as Mexico with less than 1%, may instead reflect filtering limited to specific organizations or networks. However, these aggregates are only indicative and can misattribute the underlying source of filtering. In countries such as Russia [50], censorship enforcement is decentralized, meaning that only a small fraction of ASes perform filtering despite operating under state mandate. Conversely, a single topologically central AS may create the appearance of country-wide filtering even in the absence of government involvement. Therefore, low observed AS coverage does not necessarily imply the absence of state control, and high coverage does not necessarily imply uniform nationwide enforcement.

5.4 HTTP and HTTPS Filtering

We have already shown that 19 countries actively engage in VoIP filtering by dropping app-specific STUN packets. However, there is anecdotal evidence that well-known censors like China and Iran often completely block the Instant Messaging (IM) apps [6, 25, 57]. Thus, our objective is to distinguish between selective blocking of VoIP functionality and complete blocking of IM applications.

We evaluate the web (HTTP/HTTPS) reachability of instant messaging (IM) applications using Hyperquack [49], an active HTTP(S)-based blocking detection system. We curated a dataset of 300 domains corresponding to 19 popular IM applications, including WhatsApp, Telegram, Signal, Instagram, and Facebook Messenger. We inspected the PCAPs for individual apps and identified domains containing the app name (e.g., whatsapp.com, telegram.org).

Using Hyperquack, we issue HTTP(S) probes for each domain and observe network behavior indicative of interference, such as connection resets, injected responses, or persistent timeouts. Our measurement campaign ran from January 15 to January 28, 2026, during which scans were conducted twice per week. Each scan tested the reachability of all 300 domains, producing multiple observations per domain across time. This repeated-measurement design allows us to capture both persistent and transient blocking behavior.

Our STUN-based investigation, coupled with Hyperquack results, enables us to distinguish selective VoIP blocking from complete app blocking by comparing reachability outcomes across different domain classes within the same app. When only app-specific VoIP packets are blocked, we attribute the interference to selective VoIP filtering; when messaging domains are also filtered, we infer app-wide blocking.

In our global measurement campaign, we collected about 7.56M reachability measurements targeting IM app domains distributed across 71 countries. Our analysis reveals varying levels of network interference with IM app infrastructure. We define two metrics to analyze web-filtering measurement data obtained by running HyperQuack.

Probe Filtering Rate: This metric represents the fraction of probes sent to a country that are classified as filtered. We compute this value for each domain, in every country, on a daily basis. To compute this metric, we first consider only probes for which measurements were successfully completed, and then apply the Hyperquack heuristics to classify measurements exhibiting anomalous behavior as filtered.

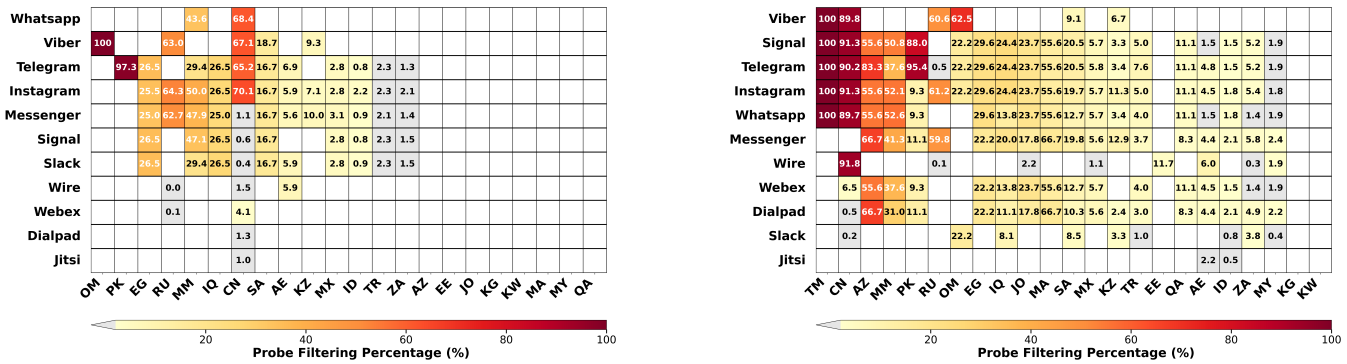
AS Filtering Rate: This metric represents the percentage of Autonomous Systems (ASes) within a country that account for probes classified as anomalous, thereby indicating how widespread filtering behavior is in that country. The computation follows the same initial steps as Probe Filtering Rate, after which filtered probe IPs are grouped by AS, and the fraction of ASes associated with filtered probes is derived.

Note that a high value of Probe Filtering Rate indicates that many users encounter filtering, whereas a low value indicates that filtering is rare or localized. However, it does not indicate how widely filtering is deployed across networks. It might happen that we observe a high probing rate, but the majority of the probes are filtered in a single or a few ASes. Hence, we compute the AS Filtering Rate, where a high value indicates filtering is present across many ISPs or networks, while a low value indicates it is concentrated in a few networks.

The results for both HTTP- and HTTPS-level domain blocking are presented in Figure 12a and Figure 12b. The color gradient represents the probe filtering rate, while the values in the individual box represent the AS filtering rate. The countries on the X-axis are sorted in decreasing order of probe filtering rates. We note multiple interesting findings.

Overall, we see much less filtering at the HTTP level than at the HTTPS level, which is understandable, as most Internet traffic is HTTPS. For HTTP, we note that Oman filters 100% of probes for Viber across all ASes, closely followed by Pakistan, with about 97.3% AS filtering rate for Telegram. China also filters a large proportion of HTTP traffic for WhatsApp, Viber, Telegram, and Instagram. For HTTPS, we observed 100% probe filtering in Turkmenistan across all ASes, followed by China for five VoIP apps. China also filters Wire, an app popular in East Asia. We also observed Russia heavily filtering Telegram and Signal.

Apart from individual country results, we observed a strong correlation between the proportion of probes filtered and the proportion of ASes filtering those probes. We quantified this relationship by computing the Pearson correlation coefficient across all countries using probe-filtering and AS-filtering rates, obtaining a high correlation of 0.96. This indicates that high probe filtering is almost always accompanied by



(a) HTTP Interference

(b) HTTPS Interference

Figure 12: VoIP Application Interference: Color gradient shows probe filtering rate, and values show AS filtering rate.

widespread filtering across ASes, and vice versa. This further suggests that, in countries where high filtering rates are observed, filtering behavior is consistently applied across most ASes, indicating the presence of coordinated or nationwide filtering policies targeting these applications.

Across both HTTP and HTTPS, many countries did not filter VoIP apps at the domain level but did filter at the STUN level. We found numerous combinations of filtering across countries. Pakistan (Signal) and Myanmar (Telegram) both block VoIP apps at the domain and STUN levels. Whereas, Kazakhstan (Telegram) and Qatar (Telegram) implemented only HTTPS-level domain and STUN filtering. Estonia (Telegram) and Kuwait (Telegram) are examples of countries that only block based on STUN. Finally, China and Russia implemented only domain-level censorship without STUN filtering.

6 Discussion

Was some of the observed censorship a collateral of keyword filtering? For some VoIP apps (e.g., Telegram and Signal), filtering is triggered by the app domain appearing in the plaintext `realm` field of STUN packets (see Table 4), namely `telegram.org` and `signal.org`. This suggests that censorship may result from general keyword filtering rather than STUN-specific rules.

To distinguish these cases, we conducted additional experiments by sending probes to target IPs that elicited a censored response: (i) UDP packets carrying the blocked domains as payload, and (ii) STUN packets with the domain placed outside the `realm` field. Neither probes triggered filtering, indicating that censorship is specific to STUN packets and relies on inspecting particular STUN header fields. The only exception is Pakistan, where an app domain name present in any attribute of the STUN packet will trigger the middlebox.

Fingerprintability of VoIP apps The censorship we identify in this study primarily targets STUN traffic. To evaluate whether this reflects a broader ability to identify VoIP applications, we analyzed packet traces from popular VoIP apps across five independent runs and identified stable, application-specific protocol features. We find that each application exposes multiple distinct fingerprints, as summarized in Table 4.

These results indicate that widely used VoIP applications are readily identifiable and can be blocked using relatively simple mechanisms. At the same time, some applications may intentionally retain distinguishable fingerprints to avoid collateral blocking in regions where competing VoIP services are restricted. Thus, anti-fingerprinting involves a tradeoff between censorship resistance and deployment considerations. However, note that relying solely on app-level changes is insufficient; protocols must be designed for resilience.

Ground Truth The most reliable way to obtain ground truth is through in-country experiments. However, like other remote measurement approaches [54, 61], *Revelio* trades direct ground-truth validation for advantages such as improved user safety and broader geographic coverage. Consequently, remote techniques must instead rely on auxiliary validation mechanisms to establish confidence in their results. *Revelio* addresses this challenge in two ways. First, our testing methodology is grounded in prior empirical observations from [41], in which all experiments were conducted from in-country vantage points (VPs). Second, *Revelio* probing methodology itself provides an inherent validation mechanism for identifying censorship. Specifically, we classify traffic as censored only when P2 probes (which preserve the application fingerprint) elicit discriminatory responses, whereas P3 probes (which obscure the fingerprint using modified STUN packets) do not trigger the same behavior. This confirmatory probing strategy allows us to distinguish fingerprint-based interference from ordinary network failures.

On using STUN Binding for detection For detecting blocking of VoIP apps, there are two distinct packets (STUN `Allocate` and STUN `Binding`) that can be fingerprinted by middleboxes. In our study, we derived all our findings from the `Allocate` packets. For `Binding` requests we mostly found

no filtering, except for Messenger and Instagram.

To perform the analysis for `Binding`, we followed the same process: first, inspecting the STUN Binding request packets generated by all the apps under consideration. For most apps, we did not find any uniquely identifiable attribute. However, a few apps had a fingerprintable Binding packet. Instagram and Messenger Binding packets contain unknown attribute `0xdaba` and a Username `EDGE:EDGE`; Teams similarly use `0x8036` as an unknown attribute, and Webex has a unique Software attribute called `Ciscospark`. We included such Binding requests in our scans, but observed filtering only in Messenger and Instagram for the UAE and Myanmar (see Section A).

Residual censorship UDP residual censorship persists after triggering and is enforced by source IP, causing cross-application disruption, with some middleboxes blocking UDP traffic entirely. Using controlled STUN probing, we measure censorship persistence and show durations cluster around fixed timer values (1–2 min.) across countries, indicating common middlebox configurations, while a few deployments exhibit much longer blocking (see Appendix Section C).

Challenges in remote censorship detection *Revelio*, like all existing remote measurement techniques, cannot detect all types of STUN filtering, e.g., if the censor targets only egress traffic. There are anecdotes that Saudi Arabia blocks VoIP calls, but *Revelio* could not detect such blocking. The absence of STUN-based WhatsApp filtering in Saudi Arabia may therefore be an artifact of this limitation. However, because *Revelio* successfully detects Telegram censorship in the same setting, we attribute the lack of observed WhatsApp blocking to other factors. First, recent reports suggest that Saudi Arabia lifted restrictions on WhatsApp calling services in February 2025 [37]. Second, we continue to observe interference in domain-based filtering of WhatsApp traffic in 12.7% of Saudi ASes (see Figure 12), indicating that non-STUN-based forms of VoIP blocking may still persist.

Explicit Comparison with Lipphardt et al. [41] While the authors demonstrated the feasibility of detecting VoIP blocking through manual analysis of packet traces collected from a limited number of vantage points, *Revelio* introduces a fundamentally different methodology grounded in active, remote measurement. Specifically, it contributes STUN-aware Internet scanning, differential probe-response analysis, and STUN traceroute techniques to systematically identify censorship behavior and localize filtering middleboxes.

Methodologically, [41] relied primarily on comparing application behavior in censored and uncensored regions, using manual packet-trace inspection to identify blocking. *Revelio* instead develops a generalized and automated detection framework capable of identifying a broader range of censorship behaviors, including packet dropping, ICMP spoofing, threshold-triggered filtering, and throttling-based interference. It also introduces the novel technique of leveraging otherwise unresponsive IP address space to infer censorship through anomalous ICMP responses.

Using these techniques, *Revelio* performs the first global-scale study of VoIP censorship across 71 countries and 11 popular VoIP applications, uncovering evidence of VoIP filtering in 19 countries, substantially expanding the geographic and application coverage of [41], which examined fewer applications in a smaller set of countries. The study additionally develops a comprehensive taxonomy of STUN-filtering middleboxes, identifying multiple classes and subtypes of censorship infrastructure not previously documented.

7 Conclusion

VoIP censorship is emerging as a pervasive yet undermeasured threat to open communication. This paper introduces *Revelio*, which enables scalable remote detection of VoIP filtering through STUN-aware scanning and robust response analysis. Across 71 countries and 11 applications, we reveal widespread, VoIP app-specific interference, diverse middlebox behaviors, and multi-layer censorship strategies. Our findings show that blocking often relies on simple, app-specific STUN fingerprints, suggesting that censorship resistance is technically feasible. We hope these results inform policymakers, empower measurement efforts, and encourage app developers to adopt designs that prioritize resilience against censorship worldwide in the future Internet ecosystem.

Ethical Considerations

Remote studies of Internet censorship interact with network infrastructure in jurisdictions where access to communication services may be deliberately restricted, potentially posing risks to maintainers of pubic network infrastructure in those regions. A substantial body of prior work has therefore developed carefully and principally designed remote censorship measurement techniques [47, 49, 53, 56, 61], that are ethically sound and safe to conduct longitudinal global measurements. *Revelio* follows these principles, which have matured over time, through careful oversight by the censorship and network measurements community.

Stakeholders and Ethical Framework: In designing our study, we explicitly considered ethical implications for all relevant stakeholders, including users in censored regions, network operators, service providers, and the broader research community, and structured our methodology to minimize potential harms while preserving the societal benefits of understanding censorship practices. Consistent with prior work and current community guidance, we designed the study in accordance with established ethical frameworks, including the Belmont Report [27] and the Menlo Report [9].

Active Internet measurements aimed at detecting censorship raise ethical concerns because they involve interacting

with live infrastructure in environments where communication restrictions may exist. We therefore evaluated potential tangible harms (e.g., operational disruption or unintended attribution) as well as risks of violating individual rights, such as exposing users to scrutiny or interfering with expected network behavior. We consulted our institution's ethics review board, which determined that the study does not involve human subjects or personally identifiable data and therefore falls outside formal review requirements. Nevertheless, we adopted community best practices to minimize risks and carefully weighed expected societal benefits against potential negative outcomes, following the Menlo principles of Beneficence, Respect for Persons, and Justice.

Measurement Design and Risk Minimization: Unlike approaches that rely on volunteers or infrastructure located within censored regions [41], our methodology operates entirely through remote measurements initiated from infrastructure under our control. Probes are directed only toward publicly reachable IP addresses and consist solely of standards-compliant STUN packets crafted to elicit observable network behavior. No end users are recruited, and hosts within censored regions are not required to access blocked content or initiate prohibited communications. This design choice reduces potential risks to individuals and avoids placing disproportionate burdens on users located in restrictive environments.

Our probes are intended to trigger responses from censorship infrastructure, which could theoretically be misinterpreted as attempts by local hosts to access restricted services. We consider this to pose no more than minimal risk for several reasons. Probe traffic originates entirely from our measurement infrastructure and differs substantially from ordinary VoIP or web communication, consisting only of minimal STUN exchanges without establishing application sessions or media flows. Our infrastructure is clearly identifiable as an academic measurement project through reverse DNS entries, WHOIS records, and a public webpage that provides project details and contact information for opt-out requests. While we did not receive any opt-out requests during our December measurement campaign, we received one request to opt out during our pilot study. We excluded the requested IPs from our scanning list. Measurements followed established best practices for Internet-wide scanning, including honoring opt-out requests and maintaining shared blocklists.

Sensitivity of Collected Data: The collected data consists of STUN server IPs and the middlebox IPs (wherever possible). But note that the addresses of the STUN servers themselves, or the IP addresses of the middleboxes, are not personal information. Further, at no point in the study was an attempt made to collect individual end clients' IPs. The probes were also launched from client machines, which were completely controlled and owned by the authors.

Decisions Regarding Consent and Fairness: Obtaining informed consent from all contacted network operators would theoretically eliminate residual concerns, but doing so at a global scale is infeasible and could itself increase risk by drawing attention to particular operators or networks. Restricting measurements only to consenting networks would also introduce significant bias and reduce the societal value of understanding censorship practices, disproportionately excluding regions where censorship is most impactful. Our approach, therefore, seeks to balance Respect for Persons and Justice principles by minimizing risks while enabling research whose primary beneficiaries include populations affected by communication restrictions.

Publication Impact Considerations: We considered the broader impacts of the publication itself. While measurement techniques could theoretically inform censorship operators, the methods rely on standard protocol behaviors already observable in deployed systems, and publication primarily enables transparency, reproducibility, and development of censorship-resilient communication systems. We therefore judge that the societal benefits of public disclosure outweigh potential risks.

Summary of Ethical Assessment: Taken together, we believe that the risks associated with our measurements are minimal and justified by the societal benefits of improving transparency and understanding of global VoIP censorship practices, while adhering to current ethical expectations for Internet measurement research.

Open Science

To promote reproducibility and support open science, we are releasing *Revelio*'s source code, and analysis scripts [69]. It has individual modules which will (1) scrape country ASNs and prefix data (`scraping/launcher.sh`), merge and clean prefixes for scanning, (2) perform protocol-level STUN probes using `Zmap`-assisted pipelines (`protocol_probing/launcher.sh`), and (3) post process `Zmap` outputs to produce per country, analysis ready results.

While we are releasing full code, data release decisions are based on lessons learned from the censorship community. For instance, remote measurement data for website blocking has been made public by large-scale projects such as CensoredPlanet. Thus, we released the complete dataset from our HyperQuack measurements. In contrast, no project currently publishes censored STUN server data; even [41] released only anonymized IPs. Blindly releasing raw IP data risks DoS scenarios where residual censorship, combined with remote triggering via source IP spoofing, could disrupt end users. Thus, we have conservatively not released STUN server IPs.

Acknowledgments

We are thankful to our shepherd and the anonymous reviewers for their insightful comments. We also gratefully acknowledge

Armin for his assistance in running HyperQuack, Sanoj for helping us measure residual censorship, Harshwardhan for modifying Zmap, and the Onyx research group for its support.

References

[1] Anna Ablove, Shreyas Chandrashekaran, Hieu Le, Ram Sundara Raman, Reethika Ramesh, Harry Oppenheimer, and Roya Ensafi. Digital discrimination of users in sanctioned states: the case of the cuba embargo. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3909–3926, 2024.

[2] Giuseppe Aceto, Antonio Montieri, and Antonio Pescapé. Internet censorship in Italy: An analysis of 3G/4G networks. In *IEEE International Conference on Communications (ICC)*, pages 1–6. IEEE, 2017.

[3] Abdulrahman Alaraj, Kevin Bock, Dave Levin, and Eric Wustrow. A global measurement of routing loops on the internet. In *International Conference on Passive and Active Network Measurement*, pages 373–399. Springer, 2023.

[4] Fatemah Alharbi, Michalis Faloutsos, and Nael Abu-Ghazaleh. Opening digital borders cautiously yet decisively: Digital filtering in saudi arabia. In *10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20)*, 2020.

[5] Alice, Bob, Carol, Jan Beznazwy, and Amir Houmansadr. How China detects and blocks shadowsocks. In *Proceedings of Internet Measurement Conference (IMC 2020)*. ACM, 2020.

[6] Aljazeera. Iran blocks Signal messaging app after WhatsApp exodus. https://www.aljazeera.com/news/2021/1/26/iran-blocks-signal-messaging-app-after-whatsapp-exodus.

[7] Anonymous, Arian Akhavan Niaki, Nguyen Phong Hoang, Phillipa Gill, and Amir Houmansadr. Triplet censors: Demystifying great firewall's DNS censorship behavior. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*. USENIX Association, August 2020.

[8] Simurgh Aryan, Homa Aryan, and J Alex Halderman. Internet censorship in Iran: A first look. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2013.

[9] M. Bailey, E. Kenneally, D. Maughan, and D. Dittrich. The menlo report. *IEEE Security & Privacy*, 10(02):71–75, mar 2012.

[10] Jan Beznazwy and Amir Houmansadr. How China detects and blocks shadowsocks. In *Proceedings of the Internet Measurement Conference*, pages 111–124, 2020.

[11] Blink Visa. Top 20 Audio Call Apps. https://blinkvisa.com/blog/secretchat/top-audio-call-apps/.

[12] Blue Chi. Voice Over IP (VOIP) Banned in Oman. https://omancommunityblog.blogspot.com/2007/10/voice-over-ip-voip-banned-in-oman.html.

[13] Kevin Bock, Yair Fax, Kyle Reese, Jasraj Singh, and Dave Levin. Detecting and evading {Censorship-in-Depth}: A case study of {Iran's} protocol whitelister. In *10th USENIX Workshop on Free and Open Communications on the Internet*, 2020.

[14] Kevin Bock, George Hughey, Xiao Qiang, and Dave Levin. Geneva: Evolving censorship evasion strategies. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 2199–2214, New York, NY, USA, 2019. Association for Computing Machinery.

[15] Abdelberi Chaabane, Terence Chen, Mathieu Cunche, Emiliano De Cristofaro, Arik Friedman, and Mohamed Ali Kaafar. Censorship in the wild: Analyzing internet filtering in Syria. In *Proceedings of Internet Measurement Conference*, pages 285–298. ACM, 2014.

[16] Zimo Chai, Amirhossein Ghafari, and Amir Houmansadr. On the importance of encrypted-sni (ESNI) to censorship circumvention. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2019.

[17] Charlie Campbell. What Authoritarians May Learn About Censorship From Nepal's Protests. https://time.com/7315858/nepal-protests-social-media-ban-authoritarian-censorship-internet-freedom-analysis/.

[18] David Ehl. Cyprus conflict: Will fresh talks bring a resolution? https://www.dw.com/en/cyprus-conflict-will-fresh-talks-bring-a-resolution/a-71965732.

[19] Arun Dunna, Ciarán O'Brien, and Phillipa Gill. Analyzing China's blocking of unpublished tor bridges. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2018.

[20] Zakir Durumeric, Eric Wustrow, and J Alex Halderman. {ZMap}: Fast internet-wide scanning and its security applications. In *22nd USENIX Security Symposium (USENIX Security 13)*, pages 605–620, 2013.

[21] Hurricane Electric. Bgp prefixes by country. https://bgp.he.net/.

[22] Roya Ensafi, David Fifield, Philipp Winter, Nick Feamster, Nicholas Weaver, and Vern Paxson. Examining how the great firewall discovers hidden circumvention servers. In *Proceedings of Internet Measurement Conference*, pages 445–458, 2015.

[23] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R Crandall. Analyzing the Great Firewall of China over space and time. *Proceedings on privacy enhancing technologies*, 2015(1):61–76, 2015.

[24] Tony Bates et al. Cidr report asv2.0. https://www.cidr-report.org/as2.0/.

[25] Euronews. Iran Maintains Block on WhatsApp and Instagram. https://www.euronews.com/2023/02/01/iran-maintains-block-on-whatsapp-and-instagram.

[26] Arturo Filasto and Jacob Appelbaum. OONI: Open observatory of network interference. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2012.

[27] National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research. The belmont report: Ethical principles and guidelines for the protection of human subjects of research, 1979.

[28] Freedom House. Israel Cannot Remain a Democracy Without Protecting Press Freedom. https://freedomhouse.org/article/israel-cannot-remain-democracy-without-protecting-press-freedom.

[29] Guru99. 12 Best FREE International Calling Apps (2026). https://www.guru99.com/best-apps-to-make-free-international-calls.html.

[30] Hafizullah Maroof and Doug Faulkner. Social media content restricted in Afghanistan, Taliban sources confirm. https://www.bbc.com/news/articles/cgrq8d0jpdwo.

[31] Freedom House. Country Freedom Score. https://freedomhouse.org/countries/freedom-world/scores, 2024.

[32] IETF. Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN). https://datatracker.ietf.org/doc/html/rfc5766, April 2010. RFC 5766.

[33] IFEX. Blocking of WhatsApp in Yemen: A Matter of Life and Death. https://ifex.org/blocking-of-whatsapp-in-yemen-a-matter-of-life-and-death/.

[34] Jill Jermyn and Nicholas Weaver. Autosonda: Discovering rules and triggers of censorship devices. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2017.

[35] Sheharbano Khattak, Mobin Javed, Philip D Anderson, and Vern Paxson. Towards illuminating a censorship monitor's model to facilitate evasion. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2013.

[36] Jeffrey Knockel, Lotus Ruan, and Masashi Crete-Nishihata. Measuring decentralization of Chinese keyword censorship via mobile games. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2017.

[37] KSA Directory. Saudi Arabia Lifts the Ban on WhatsApp Voice and Video Calls. https://www.ksa.directory/saudi-arabia-lifts-the-ban-on-whatsapp-voice-and-video-calls/1864/n.

[38] Le Monde. Freedom of Information Is Under Attack From All Sides. https://www.lemonde.fr/en/opinion/article/2024/05/06/freedom-of-information-is-under-attack-from-all-sides_6670557_23.html.

[39] Philip Levis. The collateral damage of internet censorship by DNS injection. *ACM SIGCOMM CCR*, 42(3), 2012.

[40] Life in Saudi Arabia. Why is WhatsApp Calling Banned in Saudi Arabia? https://lifeinsaudiarabia.net/why-is-whatsapp-calling-banned-in-saudi-arabia/.

[41] Friedemann Lipphardt, Anja Feldmann, and Devashish Gosain. Can you hear me? a first study of voip censorship techniques in saudi arabia and the uae. In *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*, pages 720–736. IEEE, 2025.

[42] M. Petit-Huguenin and G. Salgueiro and J. Rosenberg and D. Wing and R. Mahy and P. Matthews. Session Traversal Utilities for NAT (STUN). https://www.rfc-editor.org/rfc/rfc8489.html.

[43] Bill Marczak, Nicholas Weaver, Jakub Dalek, Roya Ensafi, David Fifield, Sarah McKune, Arn Rey, John Scott-Railton, Ronald Deibert, and Vern Paxson. China's great cannon. *Citizen Lab*, 10, 2015.

[44] Zubair Nabi. The anatomy of web censorship in pakistan. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2013.

[45] Arian Akhavan Niaki, Shinyoung Cho, Zachary Weinberg, Nguyen Phong Hoang, Abbas Razaghpanah, Nicolas Christin, and Phillipa Gill. IClab: a global, longitudinal internet censorship measurement platform. In *IEEE Symposium on Security and Privacy (SP)*, pages 135–151, 2020.

[46] Niklas Niere, Felix Lange, Nico Heitmann, and Juraj Somorovsky. Encrypted client hello (ech) in censorship circumvention. 2025.

[47] Sadia Nourin, Xi Jiang Van Tran, Kevin Bock, Nick Feamster, Nguyen Phong Hoang, and Dave Levin. Measuring and evading turkmenistan's internet censorship. In *International World Wide Web Conference (WWW)*, 2023.

[48] OONI. Ooni: Open observatory of network interference, 2025.

[49] Ram Sundara Raman, Mona Wang, Jakub Dalek, Jonathan Mayer, and Roya Ensafi. Network measurement methods for locating and examining censorship devices. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*, pages 18–34, 2022.

[50] Reethika Ramesh, Ram Sundara Raman, Matthew Bernhard, Victor Ongkowijaya, Leonid Evdokimov, Anne Edmundson, Steven Sprecher, Muhammad Ikram, and Roya Ensafi. Decentralized control: A case study of russia. In *Network and Distributed Systems Security (NDSS) Symposium*, 2020.

[51] Reporters Without Borders. Kuwait. https://rsf.org/en/country/kuwait.

[52] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing. Session traversal utilities for nat (stun). RFC 5389. IETF, 2008.

[53] Will Scott, Thomas Anderson, Tadayoshi Kohno, and Arvind Krishnamurthy. Satellite: Joint analysis of CDNs and network-level interference. In *USENIX Annual Technical Conference (ATC)*, pages 195–208, 2016.

[54] Ram Sundara Raman, Prerana Shenoy, Katharina Kohls, and Roya Ensafi. Censored planet: An internet-wide, longitudinal censorship observatory. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 49–66, 2020.

[55] Ram Sundara Raman, Adrian Stoll, Jakub Dalek, Reethika Ramesh, Will Scott, and Roya Ensafi. Measuring the deployment of network censorship filters at global scale. In *Proceedings of the Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA*, 2020.

[56] Jonas Tai, Karthik Nishanth Sengottuvelavan, Peter Whiting, and Nguyen Phong Hoang. {IRBlock}: A {Large-Scale} measurement study of the great firewall of iran. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 705–722, 2025.

[57] The Guardian. Signal: China appears to have blocked encrypted messaging app. https://www.theguardian.com/world/2021/mar/16/signal-blocked-china-encrypted-messaging-app.

[58] The Wall Street Journal. Qatar Arrests Individual for Social Media Posts. https://www.wsj.com/livecoverage/iran-war-news-updates-2026/card/qatar-arrests-individual-for-social-media-posts-yiP0fAPS9pFwyjmits25.

[59] The Washington Post. New Russian law criminalizes online searches for controversial content. https://www.washingtonpost.com/world/2025/07/17/russia-internet-censorship/?utm_source=chatgpt.com.

[60] Sebastian Usher. Saudi Arabia blocks Viber messaging service. https://www.bbc.com/news/world-middle-east-22806848, 2013.

[61] Ben VanderSloot, Allison McDonald, Will Scott, J. Alex Halderman, and Roya Ensafi. Quack: Scalable remote measurement of application-layer censorship. In *USENIX Security Symposium*, 2018.

[62] Vasilis Ververis, Tatiana Ermakova, Marios Isaakidis, Simone Basso, Benjamin Fabian, and Stefania Milan. Understanding internet censorship in europe: The case of spain. In *13th ACM Web Science Conference 2021*, pages 319–328, 2021.

[63] Vasilis Ververis, George Kargiotakis, Arturo Filasto, Benjamin Fabian, and Afentoulis Alexandros. Understanding internet censorship policy: The case of Greece. In *USENIX workshop on Free and Open Communications on the Internet*, 2015.

[64] Zhongjie Wang, Yue Cao, Zhiyun Qian, Chengyu Song, and Srikanth V Krishnamurthy. Your state is not mine: A closer look at evading stateful internet censorship. In *Proceedings of the Internet Measurement Conference*, pages 114–127, 2017.

[65] Philipp Winter and Stefan Lindskog. How the Great Firewall of China is blocking Tor. In *USENIX workshop on Free and Open Communications on the Internet (FOCI)*, 2012.

[66] Mingshi Wu, Ali Zohaib, Zakir Durumeric, Amir Houmansadr, and Eric Wustrow. A wall behind a wall: Emerging regional censorship in china. In *2025 IEEE*

Symposium on Security and Privacy (SP), pages 1363–1380. IEEE, 2025.

[67] Xueyang Xu, Z Morley Mao, and J Alex Halderman. Internet censorship in China: Where does the filtering occur? In *Proceedings of Passive and Active Network Measurement*, pages 133–142. Springer, 2011.

[68] Tarun Kumar Yadav, Akshat Sinha, Devashish Gosain, Piyush Kumar Sharma, and Sambuddho Chakravarty. Where the light gets in: Analyzing web censorship mechanisms in India. In *Proceedings of the Internet Measurement Conference 2018*, 2018.

[69] Zenodo. Revelio — Global STUN Measurement Toolkit. https://doi.org/10.5281/zenodo.20362017.

[70] Ali Zohaib, Qiang Zao, Jackson Sippe, Abdulrahman Alaraj, Amir Houmansadr, Zakir Durumeric, and Eric Wustrow. Exposing and Circumventing SNI-based QUIC Censorship of the Great Firewall of China. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 783–802, 2025.

A Filtering of Messenger/Instagram Binding packets

In countries such as the UAE and Myanmar, we observed middleboxes that dropped STUN Binding requests based on fingerprintable attributes (Unknown attribute `0xdaba` and Username `EDGE:EDGE`). We observe a drop in the number of IPs responding to Messenger Binding Requests compared to vanilla STUN Binding requests. But when we set the fingerprintable attributes to random values, we again observed a similar increase in the number of responding IPs.

B Zstun

While `Zmap` provides Internet scanning capabilities, we had to modify its functioning to adapt it for our STUN-specific scanning and filtering detection. We call this modified version `Zstun`. In particular, we sent our STUN probes (P1, P2, and P3) by specifying the UDP payload using the `-probe-args=hex:"$pkt_hex"` option. Further, `Zmap` does not parse STUN packets by default. It only informs if a UDP packet was received. However, we needed additional information about the response to characterize the filtering. We thus modified the latest `Zmap` (v4.3.4) to parse UDP-based responses and extract the STUN header and attributes in the application payload. Specifically, we modify `void udp_process_packet` and `static fielddef_t fields[]` in `src/probe_modules/module_udp.c` to add additional steps for parsing. Note that modifying the source also enables real-time response reception and attribution, eliminating the need to store and analyze them later.

App	Packet Type	Fingerprinting Features
WhatsApp	Allocate Request Allocate Success Response	Unknown attributes: `0x4000`, `0x4024`, `0x4025`, `0x4002`
Telegram	Allocate Request Allocate Error Response Create Permission Request Channel Bind Request Refresh Request	Realm: `telegram.org` Destination port: `1400` Lifetime: `60 secs`

Signal	Allocate Request
	Allocate Error Response
	Create Permission Request
		Realm:
		`turn.cloudflare.com`
		`signal.org`
		Username regex:
		`^g.{64}$`
		Lifetime: `600 secs`

Messenger Instagram	Allocate Request Allocate Error Response Allocate Success Response Create Permission Request Binding Request Binding Success Response	Unknown attributes: `0x4000`, `0x4002`, `0xcb01`, `0xdaba` Realm: `turner.facebook` Lifetime: `900 secs` Destination port: `40003`
Viber	Allocate Request Allocate Success Response	Realm: `viber.com` Lifetime: `600 secs` Destination port: `443`
Slack	Allocate Request Allocate Error Response Create Permission Request	Realm: `*.app.chime.aws` Lifetime: `600 secs`
Webex	Binding Request	Software attribute: `Cisco-Spark`

Dialpad	Allocate Request
	Allocate Error Response
	Create Permission Request
	Refresh Request
		Realm: `ubervoip.net`
		Username:
		`fstelephony`
		Software:
		`Coturn-4.5.2 'dan Eider'`

Jitsi Meet	Allocate Request Binding Success Response Create Permission Request	Realm: `coturn.jitsi.net` Destination port: `443` Username: `1766056259 (10))` Software: `ice4j.org`

Wire	Allocate Request
	Allocate Error Response
	Binding Success Response
	Create Permission Request
		Realm: `dummy.io`
		Software:
		`Coturn-wireapp/4.6.2j`
		Username pattern:
		`d=*.v=1.k=0.t=s.*`

Teams	Allocate Request	Malformed/non-compliant STUN packets

Table 4: Application-specific fingerprintable attributes in STUN/TURN packets.

C Residual Time for Censorship Detection

Residual censorship is the duration for which censorship remains active after it has been triggered, even in the absence of further triggering traffic. Prior measurement studies have primarily examined this behavior for TCP traffic, where middleboxes often maintain a Transmission Control Block (TCB) derived from the 3-way handshake to track connection state. In contrast, our study measures residual censorship for UDP-based communication, which lacks connection state, making persistent blocking behavior more surprising and operationally significant.

Our experiments show that once censorship is triggered, affected middleboxes do not track flows by source port; instead, they block solely based on the client's source IP address. Consequently, after triggering censorship using a single UDP flow, subsequent STUN packets from the same source IP are dropped regardless of the source port or application generating the traffic.